

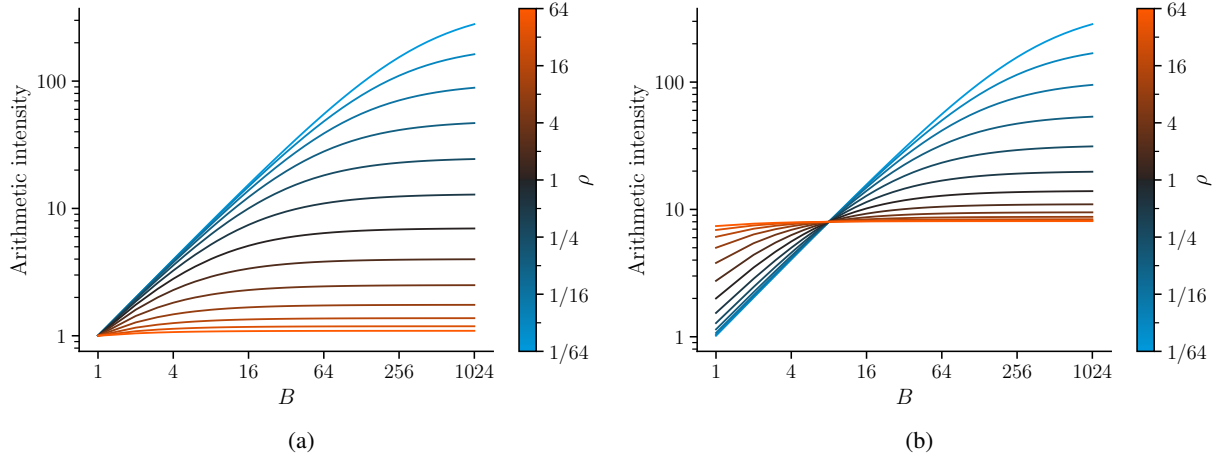


Figure C1: Relationship between $\rho = S/(gd_m)$, B and arithmetic intensity. (a) Multi-head attention, $g = 1$. (b) Grouped-query attention, $g = 8$. This highlights the importance of ρ , even with large batch size and GQA.

Model	g	d_m	S	$\rho = S/(gd_m)$	Max $\mathcal{A}/\mathcal{M}$
Llama 2 7B	1	4096	4096	1	7
Llama 2 70B	8	8192	4096	1/16	104
Llama 2 70B	8	8192	16384	1/4	32

The general relationship between ρ and arithmetic intensity is shown in Figure C1.

Hardware Properties of selected machine learning hardware.⁴ Note that $r_{\mathcal{A}}$ is the number of multiply-adds per second and $r_{\mathcal{M}}$ the number of data elements transferred per second.

Name	Memory technology	$r_{\mathcal{A}}/10^{12}$	$r_{\mathcal{M}}/10^{12}$	$r_{\mathcal{A}}/r_{\mathcal{M}}$
Bow IPU (FP16)	SRAM	175	5.5	32
A10 GPU (INT8)	GDDR	125	0.6	210
H100 SXM GPU (FP8)	HBM	990	3.35	295

Comparing $r_{\mathcal{A}}/r_{\mathcal{M}}$ for this hardware to the arithmetic intensity achievable for standard transformer models, it’s clear that sequence generation will hit a data transfer bottleneck.

In summary, we have seen that sequence generation exhibits a large-batch arithmetic intensity of just 7 for multi-head attention with $S = d_m$, up to 100 for grouped-query attention with $S \ll d_m$, while ML hardware can provide $r_{\mathcal{A}}/r_{\mathcal{M}} > 200$.

D. Measuring Time Spent in Attention

Deriving exact measurements of how long is spent in attention layers in optimised inference libraries such as `llama.cpp` and `vLLM` can be non-trivial, due to limited existing tooling in their implementations and (to a lesser extent) probing models during inference may impact performance. `llama.cpp` features a benchmarking tool called `llama-bench`, which measures the time it takes to either prefill a model with a prompt of certain length, or autoregressively generate a sequence.

We employ `llama-bench`’s existing functionality to calculate the approximate time that is spent in attention, by observing that when generating a single token, the compute and data transfer associated with the attention layers scale linearly with the

⁴For IPU (Graphcore, 2023), we use the exchange memory bandwidth of 11 TB/s. A10 (NVIDIA, 2022). H100 (NVIDIA, 2023).

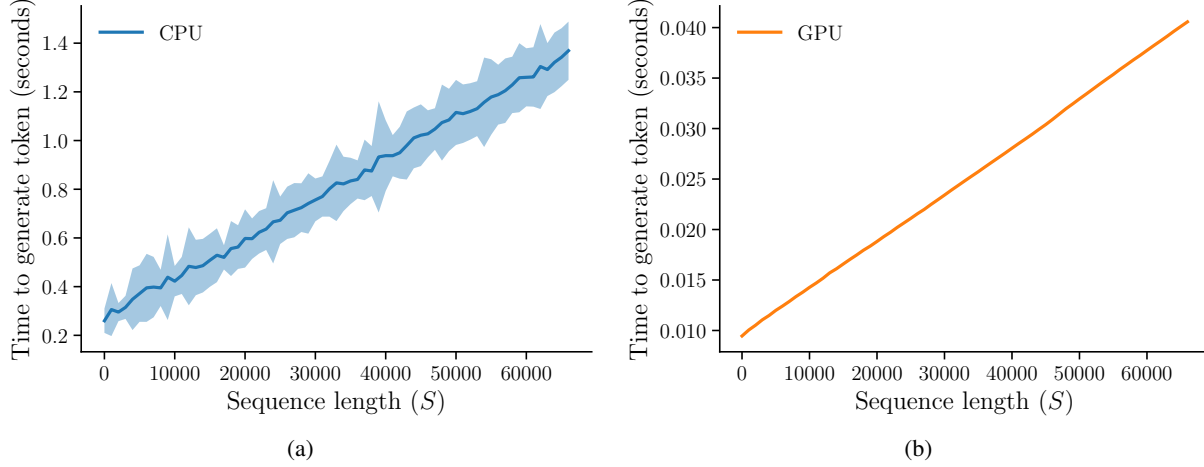


Figure D1: For each sequence length S , we prefill the KV cache with a prompt of length S before measuring the time it takes to generate a single additional token with Llama 2 7B (batch size 1). We report the mean and standard deviation over 200 runs per sequence length, for (a) CPU and (b) A100 GPU. Measurements were carried out via the `llama-bench` tooling from `llama.cpp`.

sequence length, with all other costs remaining constant. This can be seen in Figure D1, which shows the measured time it takes to generate a single token, given an existing sequence length S . As S increases, the time it takes to generate a single token scales linearly.

From the measured benchmarks, lines of best fit were computed over the interquartile range of each S (to reduce variance) for each hardware platform, which were found to be

$$y_{\text{CPU}}(S) = (1.62 \times 10^{-5})S + 0.2438 \quad (\text{D1})$$

and

$$y_{\text{GPU}}(S) = (4.686e \times 10^{-7})S + 0.009481 \quad (\text{D2})$$

for CPU and GPU respectively. The value of $y_{\text{XPU}}(0)$ corresponds to the time it takes for all non-attention data transfer and operations. Therefore, the proportion of time spent in attention, $z_{\text{XPU}}(S)$, can be approximated as

$$z_{\text{XPU}}(S) \approx \frac{y_{\text{XPU}}(S) - y_{\text{XPU}}(0)}{y_{\text{XPU}}(S)}, \quad (\text{D3})$$

the results of which can be seen in Figure 3.

E. Attention Sparsity Analysis

In order to understand how to approximate attention in pretrained transformers, we analysed the queries, values and intermediate *scores* vector (softmax output). We took 40 examples from our SQuAD 1-shot task, and generated the first completion token using the dense Llama 2 7B model, capturing the $\mathbf{q}$ vector and $\mathbf{K}$, $\mathbf{V}$ matrices from every layer and attention head, showing derived statistics in Figures 4, E1 and E2.

In Figures 4c and 4d we show that elements of the query vectors are not normally distributed, but have high sample kurtosis values. If compared to a normal distribution, the combined mass of the elements with absolute z -score exceeding 3.0 is up to $20\times$ higher. This leads us to theorise that query vectors in a pre-trained model inherently encode information **sparsely** using the tails. Therefore, the magnitude based sparsity we induce in the first stage of the algorithm does not significantly harm the approximation of the attention mappings.

We validate this claim by comparing the correspondence between the exact and approximated attention scores. SparQ Attention uses the approximate attention scores to only choose the tokens that are important for the next generation step. The

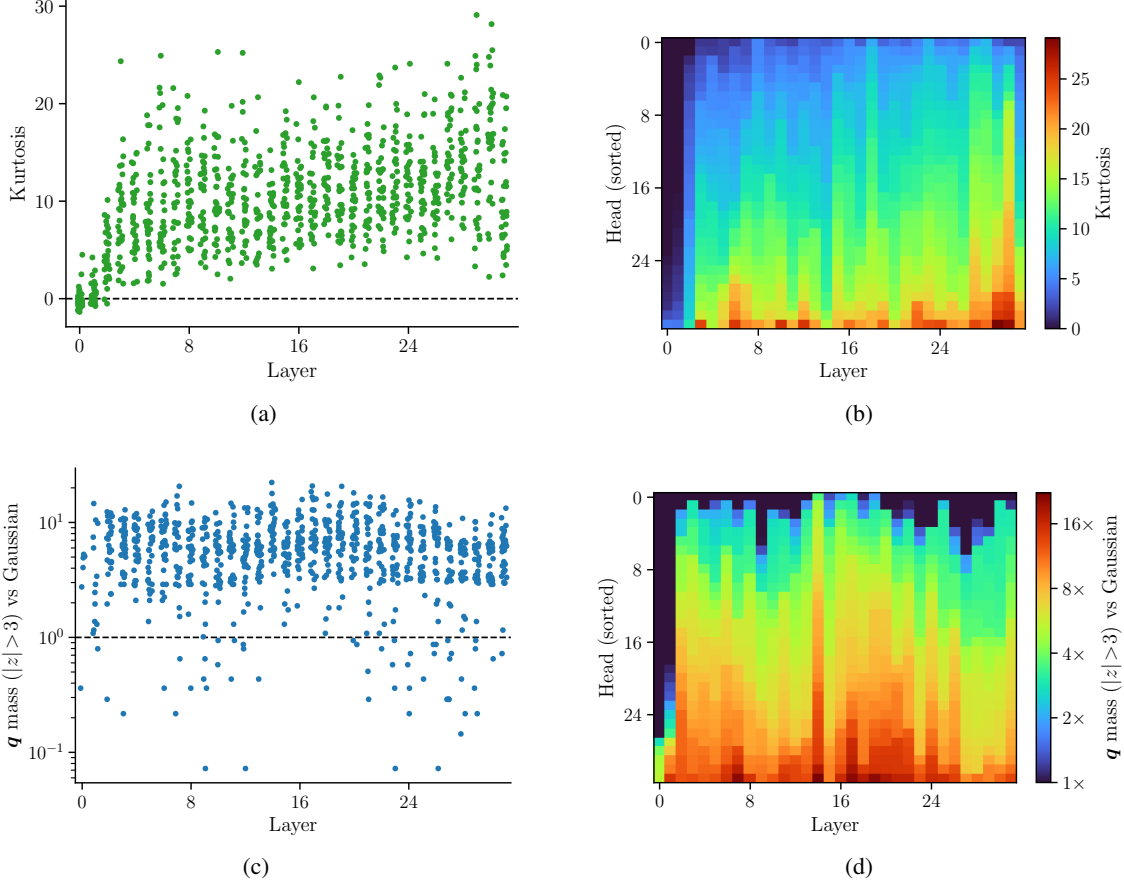


Figure E1: Statistics of components of q for each head, as a function of layer. (Top) Kurtosis (Fisher), indicating that most heads have heavy-tailed q . (Bottom) z -value mass, normalised by that of a Gaussian (0.3%), showing that most heads are outlier-heavy. All Llama 2 7B, measured over 40 SQuAD examples.

actual values of the approximate scores are not relevant, as these scores are not multiplied with value vectors and thus the property of interest to us is whether the top- k indices in the approximate scores match those of the exact counterpart. This can be measured on a scale from 0 to 1, where 1 means top- k indices are identical between the approximation and the exact scores and 0 means these sets do not overlap. We call this measure *top- k correspondence*. Figure E2 provides an overview how the choice of rank and k affects the top- k correspondence aggregated over all attention heads of the model. We see that the query vector sparsity of 50% and 75% maintain high top- k correspondence to the exact attention scores, which is consistently maintained over various values of k . Further analysis and a more detailed examination of top- k correspondence is presented in Appendix E.

It is useful to drop positions in V given attention scores, but this can save at most half of the data transfers, since the whole of K is needed to calculate these scores. We propose approximating these scores using a subset of the components of K . To test such an approximation, we measure the proportion of *overlap* between the top 32 positions in the approximated and true scores. If overlap is high, we can use the approximation to avoid transferring the whole K matrix, instead only transferring some components of K for all positions, then all components of K for some positions.

Our hypothesis is that the r largest-magnitude components of q are most useful to predicting the score, $qK^\top$. The coverage of this technique against an arbitrary-component baseline is shown in Figure E2. These results show that it is possible to achieve reasonably high overlap even using $r = d_h/8$, but that some later layers are harder to predict. Using the top- r components outperforms the first r baseline considerably.